Lab 15

Deploy WSO2 API Manager Pattern 1 (Active-Active)

WSO2 API Manager 4.7.0 · Practical exercise

Before You Begin

URLs in this guide use placeholders. Replace them with the values of the API Manager you are using:

- **<APIM_HOST>** – host name or IP address of the API Manager server (localhost when API Manager runs on your own laptop).
- **<HTTPS_PORT>** – API Manager HTTPS port: 9443 plus any port offset (Publisher, Developer Portal, Admin Portal, Management Console, token endpoint).
- **<GW_HTTPS_PORT> / <GW_HTTP_PORT>** – API Gateway ports: 8243 (HTTPS) and 8280 (HTTP) plus any port offset.
- **<APIM_HOME>** – the folder where API Manager is extracted.

The servers use a self-signed certificate by default, so the browser shows a security warning the first time: choose Advanced and continue.

- **<NODE1_HOME> / <NODE2_HOME>** – the folders of the two API Manager nodes in this lab.
- **<NGINX_CONF>** – the Nginx configuration folder (macOS: /opt/homebrew/etc/nginx, Linux: /etc/nginx, Windows: <NGINX_HOME>\conf).

Training Objective

Learn how to set up two all-in-one WSO2 API Manager nodes as an active-active pair behind an Nginx load balancer, sharing one MySQL database.

Business Scenario

PizzaShack needs a highly available API Manager: if one node stops, the portals and the gateway keep working through the other node.

High-Level Steps

- Install the prerequisites (MySQL 8, Nginx, OpenSSL) and add the host names
- Create an SSL certificate for Nginx and configure Nginx
- Create the MySQL databases and run the WSO2 scripts
- Configure node 1, copy it to node 2 and adjust node 2
- Start both nodes and validate the cluster

Before You Start

Both nodes run on your laptop: node 1 with no port offset (HTTPS 9443, gateway 8243) and node 2 with port offset 1 (HTTPS 9444, gateway 8244). Stop every other WSO2 server first (API Manager and Identity Server from earlier labs use the same ports). Users reach the portals at `https://api.am.wso2.com` and the gateway at `https://gw.am.wso2.com`, both through Nginx.

CC BY 4.0

Component	Node 1	Node 2
Port offset	0	1
Portals (HTTPS)	`9443`	`9444`
Gateway (HTTPS)	`8243`	`8244`
JMS broker (rate limiting)	`5672`	`5673`
Traffic Manager events (TCP / SSL)	`9611 / 9711`	`9612 / 9712`

Part 1 – Prerequisites and Host Names

1. MySQL 8 server (note the root password; this lab uses `root / root`), MySQL Workbench or the mysql command-line client, and the MySQL JDBC driver `mysql-connector-j-8.2.0.jar` (in the Day 3 lab files).
2. Nginx. macOS: `brew install nginx`. Linux: `sudo apt install nginx`. Windows: download the stable zip from nginx.org and extract it.
3. OpenSSL (included in macOS and Linux; on Windows it comes with Git for Windows, in Git Bash).
4. A fresh copy of the API Manager 4.7.0 zip, extracted to a new folder (not the one used in earlier labs). This is node 1, `<NODE1_HOME>`.
5. Add this line to the hosts file (macOS / Linux: `/etc/hosts`; Windows: `C:\Windows\System32\drivers\etc\hosts`; edit as administrator):

```
127.0.0.1    api.am.wso2.com    gw.am.wso2.com
```

Part 2 – Configure Nginx

6. Create the folder `<NGINX_CONF>/ssl`, go into it and create a self-signed certificate for both host names:

macOS / Linux / Git Bash

```
cd <NGINX_CONF>/ssl
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout nginx.key -out nginx.crt -subj "/CN=api.am.wso2.com" \
  -addext "subjectAltName=DNS:api.am.wso2.com,DNS:gw.am.wso2.com"
```

Example – macOS / Linux / Git Bash:

```
cd /opt/homebrew/etc/nginx/ssl
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout nginx.key -out nginx.crt -subj "/CN=api.am.wso2.com" \
  -addext "subjectAltName=DNS:api.am.wso2.com,DNS:gw.am.wso2.com"
```

CC BY 4.0

7. Create the folder for the Nginx logs used below (`<NGINX_CONF>/logs/am`, or change the log paths). Then add the configuration below: on macOS as `<NGINX_CONF>/servers/am.conf`, on Linux as `<NGINX_CONF>/conf.d/am.conf`, on Windows inside the `http { }` block of `<NGINX_CONF>\nginx.conf`. Replace `<NGINX_CONF>` with the real path (on Windows use forward slashes, for example `C:/nginx/conf`).

am.conf

```
upstream sslapi.am.wso2.com {
    server 127.0.0.1:9443;
    server 127.0.0.1:9444;
    ip_hash;
}
upstream sslgw.am.wso2.com {
    server 127.0.0.1:8243;
    server 127.0.0.1:8244;
}
server {
    listen 80;
    server_name api.am.wso2.com;
    rewrite ^/(.*) https://api.am.wso2.com/$1 permanent;
}
server {
    listen 443 ssl;
    server_name api.am.wso2.com;
    proxy_set_header X-Forwarded-Port 443;
    ssl_certificate <NGINX_CONF>/ssl/nginx.crt;
    ssl_certificate_key <NGINX_CONF>/ssl/nginx.key;
    location / {
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Server $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $http_host;
        proxy_read_timeout 5m;
        proxy_send_timeout 5m;
        proxy_pass https://sslapi.am.wso2.com;
    }
    access_log <NGINX_CONF>/logs/am/access.log;
    error_log <NGINX_CONF>/logs/am/error.log;
}
server {
    listen 443 ssl;
    server_name gw.am.wso2.com;
    proxy_set_header X-Forwarded-Port 443;
    ssl_certificate <NGINX_CONF>/ssl/nginx.crt;
    ssl_certificate_key <NGINX_CONF>/ssl/nginx.key;
    location / {
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Server $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $http_host;
        proxy_read_timeout 5m;
        proxy_send_timeout 5m;
        proxy_pass https://sslgw.am.wso2.com;
    }
    access_log <NGINX_CONF>/logs/am/access.log;
    error_log <NGINX_CONF>/logs/am/error.log;
}
```

ip_hash keeps each browser on the same node for the portals (session stickiness). The gateway needs no stickiness, so calls are spread over both nodes.

8. Check the configuration, then start Nginx (or reload it if it is already running). On macOS and Linux, ports 80 and 443 need sudo:

macOS / Linux

```
sudo nginx -t
sudo nginx
sudo nginx -s reload
```

CC BY 4.0

Windows (Command Prompt)

```
cd <NGINX_HOME>
nginx -t
start nginx
nginx -s reload
```

Part 3 – Create the Databases

9. Connect to MySQL as root and create the two databases. API Manager needs a case-sensitive collation, so use latin1_bin:

```
CREATE DATABASE wso2am_db CHARACTER SET latin1 COLLATE latin1_bin;
CREATE DATABASE wso2shared_db CHARACTER SET latin1 COLLATE latin1_bin;
```

10. Run the WSO2 scripts on them: `dbscripts/apimgt/mysql.sql` on wso2am_db and `dbscripts/mysql.sql` on wso2shared_db. In MySQL Workbench, open each script, select the database with `USE` and run it; or use the command line:

macOS / Linux

```
mysql -u root -p wso2am_db < <NODE1_HOME>/dbscripts/apimgt/mysql.sql
mysql -u root -p wso2shared_db < <NODE1_HOME>/dbscripts/mysql.sql
```

Example – macOS / Linux:

```
mysql -u root -p wso2am_db < ~/apim-node1/wso2am-4.7.0/dbscripts/apimgt/mysql.sql
mysql -u root -p wso2shared_db < ~/apim-node1/wso2am-4.7.0/dbscripts/mysql.sql
```

Windows (Command Prompt)

```
mysql -u root -p wso2am_db < <NODE1_HOME>\dbscripts\apimgt\mysql.sql
mysql -u root -p wso2shared_db < <NODE1_HOME>\dbscripts\mysql.sql
```

Example – Windows (Command Prompt):

```
mysql -u root -p wso2am_db < C:\apim-node1\wso2am-4.7.0\dbscripts\apimgt\mysql.sql
mysql -u root -p wso2shared_db < C:\apim-node1\wso2am-4.7.0\dbscripts\mysql.sql
```

Part 4 – Configure Node 1

11. Copy `mysql-connector-j-8.2.0.jar` to `<NODE1_HOME>/repository/components/lib`.

12. Edit `<NODE1_HOME>/repository/conf/deployment.toml`. In the `[server]` section, change the hostname to:

```
hostname = "api.am.wso2.com"
```

CC BY 4.0

13. Replace the two H2 database sections (`[database.apim_db]` and `[database.shared_db]`) with:

```
[database.apim_db]
type = "mysql"
url = "jdbc:mysql://localhost:3306/wso2am_db?useSSL=false"
username = "root"
password = "root"
driver = "com.mysql.cj.jdbc.Driver"

[database.shared_db]
type = "mysql"
url = "jdbc:mysql://localhost:3306/wso2shared_db?useSSL=false"
username = "root"
password = "root"
driver = "com.mysql.cj.jdbc.Driver"
```

useSSL=false and the root account keep the lab simple. In production, use a dedicated database user and an SSL connection to MySQL.

14. In the `[[apim.gateway.environment]]` section, change the four endpoint lines to the gateway host name:

```
ws_endpoint = "ws://gw.am.wso2.com:9099"
wss_endpoint = "wss://gw.am.wso2.com:8099"
http_endpoint = "http://gw.am.wso2.com"
https_endpoint = "https://gw.am.wso2.com"
```

15. Generate one internal encryption key and keep it. Both nodes must use exactly the same key: API Manager encrypts internal data with it, and if each node had its own key, the nodes could not read each other's data.

macOS / Linux / Git Bash

```
openssl rand -hex 32
```

Windows PowerShell

```
-join ((1..32) | ForEach-Object { '{0:x2}' -f (Get-Random -Maximum 256) })
```

CC BY 4.0

16. Add these sections at the end of the file. Use your key, and keep the port numbers exactly as shown (node 1 uses 5672, 9611 and 9711; node 2 uses 5673, 9612 and 9712):

```
[transport.https.properties]
proxyPort = 443

[encryption]
key = "<YOUR_64_CHARACTER_HEX_KEY>"

[apim.cache_invalidation]
enabled = true

[apim.throttling]
event_duplicate_url = ["tcp://127.0.0.1:5673"]
throttle_decision_endpoints = ["tcp://localhost:5672"]

[[apim.throttling.url_group]]
traffic_manager_urls = ["tcp://127.0.0.1:9611"]
traffic_manager_auth_urls = ["ssl://127.0.0.1:9711"]
type = "loadbalance"

[[apim.throttling.url_group]]
traffic_manager_urls = ["tcp://127.0.0.1:9612"]
traffic_manager_auth_urls = ["ssl://127.0.0.1:9712"]
type = "loadbalance"
```

Setting	Why
`proxyPort = 443`	The portals are reached through Nginx on port 443, so redirects use 443 instead of 9443.
`[encryption] key`	Same internal encryption key on both nodes.
`cache_invalidation`	When one node changes an API or policy, the other node clears its cache.
`event_duplicate_url`	Sends rate limiting decisions to the broker of the other node.
`url_group (x2)`	Each gateway sends rate limiting events to the Traffic Manager of both nodes.

17. Import the Nginx certificate into the node's truststore. The gateway reads the token signing keys through `https://api.am.wso2.com`, so it must trust the Nginx certificate; without this, every API call fails with HTTP 500 (`PKIX path building failed` in the log):

macOS / Linux

```
keytool -importcert -noprompt -alias nginx \
  -file <NGINX_CONF>/ssl/nginx.crt -storepass wso2carbon \
  -keystore <NODE1_HOME>/repository/resources/security/client-truststore.jks
```

Example – macOS / Linux:

```
keytool -importcert -noprompt -alias nginx \
  -file /opt/homebrew/etc/nginx/ssl/nginx.crt -storepass wso2carbon \
  -keystore ~/apim-node1/wso2am-4.7.0/repository/resources/security/client-truststore.jks
```

Windows (Command Prompt)

```
keytool -importcert -noprompt -alias nginx ^
  -file <NGINX_CONF>\ssl\nginx.crt -storepass wso2carbon ^
  -keystore <NODE1_HOME>\repository\resources\security\client-truststore.jks
```

Example – Windows (Command Prompt):

```
keytool -importcert -noprompt -alias nginx ^
  -file C:\nginx\conf\ssl\nginx.crt -storepass wso2carbon ^
  -keystore C:\apim-node1\wso2am-4.7.0\repository\resources\security\client-truststore.jks
```

CC BY 4.0

Part 5 – Configure Node 2

18. Before starting node 1, copy the whole `<NODE1_HOME>` folder to a new folder. This is node 2, `<NODE2_HOME>`.

19. In `<NODE2_HOME>/repository/conf/deployment.toml`, add `offset = 1` under `[server]`, and change the two `[apim.throttling]` lines to:

```
event_duplicate_url = ["tcp://127.0.0.1:5672"]
throttle_decision_endpoints = ["tcp://localhost:5673"]
```

Keep everything else the same, including the encryption key and the two url_group sections.

Part 6 – Start and Validate

20. Start node 1, then node 2 (in a second terminal):

macOS / Linux

```
cd <NODE1_HOME>/bin
sh api-manager.sh
```

Example – macOS / Linux:

```
cd ~/apim-node1/wso2am-4.7.0/bin
sh api-manager.sh
```

Windows (Command Prompt)

```
cd <NODE1_HOME>\bin
api-manager.bat --run
```

Example – Windows (Command Prompt):

```
cd C:\apim-node1\wso2am-4.7.0\bin
api-manager.bat --run
```

Use the same commands with `<NODE2_HOME>` for node 2. Until node 2 is up, node 1 logs `Cannot borrow client for ssl://127.0.0.1:9712`: that is expected and stops when node 2 starts.

21. Open the Publisher through Nginx (accept the certificate warning) and log in as admin. The portals redirect to api.am.wso2.com without a port, which shows that the reverse proxy settings work:

```
https://api.am.wso2.com/publisher
```

22. Create and publish an API (for example PizzaShack with the back end below), deploy it to the Default gateway, subscribe an application in `https://api.am.wso2.com/devportal` and generate an access token.

Endpoint

```
https://localhost:9443/am/sample/pizzashack/v1/api/
```

CC BY 4.0

23. Call the API several times through the load balancer. Each call returns HTTP 200, and Nginx spreads the calls over both nodes (see the Nginx access log):

macOS / Linux

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>" \
  "https://gw.am.wso2.com/pizzashack/1.0.0/menu"
```

Example – macOS / Linux:

```
curl -k -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" \
  "https://gw.am.wso2.com/pizzashack/1.0.0/menu"
```

Windows (Command Prompt)

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "https://gw.am.wso2.com/pizzashack/1.0.0/menu"
```

Example – Windows (Command Prompt):

```
curl -k -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" ^
  "https://gw.am.wso2.com/pizzashack/1.0.0/menu"
```

24. Both nodes serve the API because they share the database. Check each gateway directly (8243 is node 1, 8244 is node 2):

macOS / Linux

```
curl -k -H "Host: gw.am.wso2.com" \
  -H "Authorization: Bearer <ACCESS_TOKEN>" \
  "https://localhost:8243/pizzashack/1.0.0/menu"
curl -k -H "Host: gw.am.wso2.com" \
  -H "Authorization: Bearer <ACCESS_TOKEN>" \
  "https://localhost:8244/pizzashack/1.0.0/menu"
```

Example – macOS / Linux:

```
curl -k -H "Host: gw.am.wso2.com" \
  -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" \
  "https://localhost:8243/pizzashack/1.0.0/menu"
curl -k -H "Host: gw.am.wso2.com" \
  -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" \
  "https://localhost:8244/pizzashack/1.0.0/menu"
```

Windows (Command Prompt)

```
curl -k -H "Host: gw.am.wso2.com" ^
  -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "https://localhost:8243/pizzashack/1.0.0/menu"
curl -k -H "Host: gw.am.wso2.com" ^
  -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "https://localhost:8244/pizzashack/1.0.0/menu"
```

Example – Windows (Command Prompt):

```
curl -k -H "Host: gw.am.wso2.com" ^
  -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" ^
  "https://localhost:8243/pizzashack/1.0.0/menu"
curl -k -H "Host: gw.am.wso2.com" ^
  -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" ^
  "https://localhost:8244/pizzashack/1.0.0/menu"
```

CC BY 4.0

25. Stop node 1 (Ctrl+C in its terminal). `https://api.am.wso2.com/publisher` still opens, served by node 2. Calls to the PizzaShack API now fail with a back-end error, because the sample back end runs inside node 1; an API with an external back end keeps working.

Expected Outcome

Two all-in-one API Manager nodes run as an active-active pair behind Nginx, sharing one MySQL database and one encryption key. The portals and the gateway are reached through api.am.wso2.com and gw.am.wso2.com, and keep working when one node stops.

CC BY 4.0